Universidad de Las Palmas de Gran Canaria
Grado en Ciencia e Ingeniería de Datos

Big Data - Academic Year 2026/2027

Parallel Matrix Multiplication with OpenMP

Individual Assignment 3

Sara Carrizosa Tomé

GitHub Repository

November 2026

Abstract

This experiment investigates how the number of threads affects the performance of dense matrix multiplication in C. A sequential implementation was compared with an equivalent parallel version using OpenMP. Five matrix sizes and six thread configurations were tested, measuring execution time, memory usage, speedup and parallel efficiency.

The results showed that parallel execution considerably reduced runtime for larger matrices. For matrices of size 1000, the highest speedup was approximately 7.24 with 22 threads. However, increasing the thread count did not always improve performance, particularly for smaller matrices. Parallel efficiency also decreased as more threads were used. The experiment illustrates both the advantages and limitations of shared-memory parallelisation on the tested hardware.

1 Introduction

Matrix multiplication is widely used in scientific computing, machine learning and data processing. Although its basic implementation is straightforward, the number of operations increases rapidly with matrix size, making performance an important consideration.

Previous assignments examined basic multiplication in different programming languages and explored optimisation techniques. In Assignment 2, changing the loop order improved memory access patterns and reduced execution time for larger matrices.

This assignment extends that work by investigating parallel execution. Rather than changing the algorithm itself, the computation is distributed across multiple threads using OpenMP.

The main research question is how the number of threads affects execution time, memory consumption and parallel efficiency for dense matrix multiplication.

It was expected that larger matrices would benefit more from parallelisation, while smaller matrices might show limited improvements because of the overhead associated with managing threads.

2 Background

2.1 Parallel Matrix Multiplication

The standard multiplication of two dense square matrices requires three nested loops and has a time complexity of $O(n^3)$.

In the sequential implementation, each output element is calculated using the contributions of a row from matrix A and a column from matrix B.

Parallel execution allows different parts of this computation to be performed simultaneously. In this experiment, OpenMP was used to distribute the iterations of the outermost loop among multiple threads [1].

Each thread processes different rows of the output matrix. Since the threads write to separate rows, they do not need to update the same output elements concurrently.

The matrices are shared between threads, avoiding the need to create a complete copy of the input data for each worker.

2.2 Speedup and Efficiency

Two metrics were used to evaluate parallel performance.

Speedup measures how much faster the parallel implementation is compared with the sequential baseline:

$$S(p) = \frac{T_{\text{seq}}(n)}{T_p(n)} \tag{1}$$

where $T_{\text{seq}}(n)$ is the sequential execution time and $T_p(n)$ is the parallel execution time using p threads.

Parallel efficiency expresses the speedup relative to the number of threads:

$$E(p) = \frac{S(p)}{p} \tag{2}$$

Both metrics were calculated using median execution times.

Ideally, speedup would increase proportionally with the number of threads. In practice, thread management, synchronisation, memory access and hardware characteristics can limit the improvement.

3 Methodology

3.1 Experimental Environment

The experiments were conducted on a computer running Windows 11 Home, equipped with an Intel Core Ultra 7 155H processor and 32 GiB of RAM.

The processor has 16 physical cores and 22 logical processors, with a hybrid architecture combining performance, efficient and low-power efficient cores [2].

Both implementations were written in C and compiled using GCC 16.2.0 through MSYS2 UCRT64.

The sequential implementation was compiled using `-O2 -std=c11`, while the parallel version also included `-fopenmp`.

Python 3.12 was used to process the experimental results. Matplotlib 3.10.8 was used to generate the figures, and psutil 7.1.0 was used for process memory measurements.

No explicit processor affinity was configured. The operating system therefore controlled thread placement, which may have influenced the results.

3.2 Implementations

Two implementations were compared.

The sequential version reused the loop-reordered algorithm from Assignment 2, using the i-k-j loop order.

The parallel version applied the OpenMP `parallel for` directive to the outermost loop. Apart from this change, both versions used the same multiplication algorithm and data representation.

The matrices were stored in row-major order using 32-bit integers. Output initialisation was performed sequentially in both implementations.

The parallel version was tested with 1, 2, 4, 8, 16 and 22 threads. The number of threads was controlled through the `OMP_NUM_THREADS` environment variable.

No explicit OpenMP scheduling policy was specified, so the implementation used the default scheduling behaviour of GCC's OpenMP runtime. The work was distributed across iterations of the outermost loop, with each thread processing different rows of the result matrix. Since the processor contains cores with different performance characteristics, an uneven distribution of execution time between threads could affect overall performance.

All configurations were executed on the same computer. The 22-thread configuration matched the number of logical processors available on the machine. Since no configuration exceeded this limit, the experiments did not introduce oversubscription in terms of the requested thread count.

3.3 Input Data and Correctness

Five square matrix sizes were selected: 100, 300, 500, 750 and 1000.

The same input matrices from the previous assignments were reused. They contained integer values between 0 and 9, generated using Python with fixed seeds of 42 and 43 for matrices A and B, respectively.

The input matrices were stored in CSV files and loaded before the timed multiplication.

All calculations used 32-bit signed integers. Since the input values ranged from 0 to 9 and the largest matrix size was 1000, the maximum possible value of an output element was 81,000. This is well below the maximum value supported by a 32-bit signed integer, so integer overflow was not a concern in the tested configurations.

Correctness was checked using small matrices with known results, identity matrices and zero matrices.

The results of the experimental configurations were also compared element by element against previously generated reference matrices.

All tested configurations passed validation.

3.4 Benchmarking Procedure

Execution time was measured using `clock_gettime(CLOCK_MONOTONIC)`.

Input loading and matrix allocation were excluded from the timed region. Each multiplication included output initialisation and computation, while correctness checks were performed outside the measurements.

One warm-up execution was followed by five measured repetitions for each configuration.

The median was used as the main execution time metric, while the interquartile range (IQR) described variability. Individual measurements were saved in CSV files.

Speedup and efficiency were calculated using the median times for the sequential and parallel implementations at each matrix size.

Memory usage was analysed separately. The estimated storage of matrices A, B and C was calculated from their dimensions and integer data type. For the largest matrix size ($n = 1000$), the three matrices required approximately 11.44 MiB in total, assuming 32-bit integers. This estimate does not include additional memory allocated by the program or the OpenMP runtime.

Peak process Resident Set Size (RSS) was also measured using Python's psutil library [3]. Each configuration was executed in a separate process, and memory was sampled approximately every 1 ms.

These memory measurements include the runtime environment and other allocations. They may also miss short-lived peaks between samples.

The largest tested size was 1000 by 1000. All configurations completed successfully without memory allocation failures or timeouts. No formal time or memory budget was established before the experiments, and larger sizes were not investigated. Therefore, the maximum feasible problem size was not determined.

4 Results

4.1 Execution Time

Figure 1 compares the median execution times of the sequential and parallel implementations.

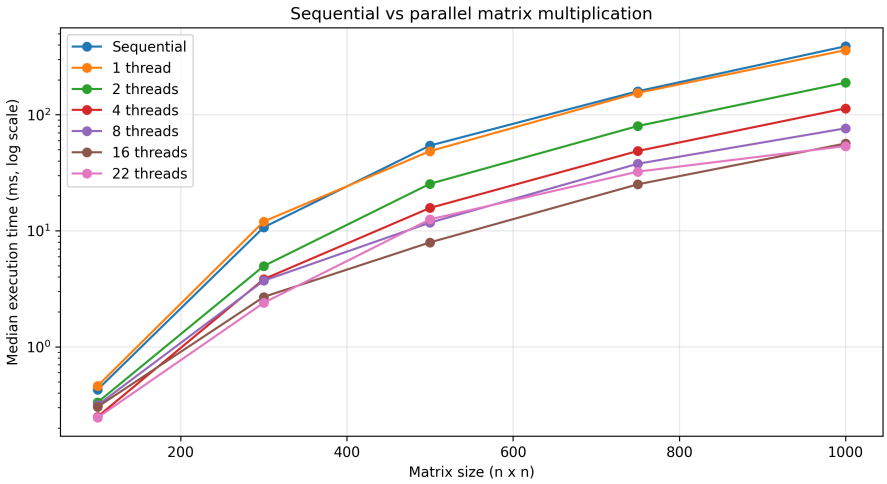


Figure 1: Median execution time for sequential and parallel matrix multiplication.

Execution time increased with matrix size for every configuration. However, the parallel implementations generally performed better as the matrices became larger.

For size 1000, the sequential implementation required approximately 389 ms, compared with 53.68 ms using 22 threads.

The difference was much smaller for the smallest matrices, where parallelisation overhead represented a larger proportion of the execution time.

Table 1 presents the results for the largest tested size.

Table 1: Performance results for 1000 x 1000 matrices.

Configuration	Median (ms)	IQR (ms)	Speedup	Efficiency (%)
Sequential	388.85	5.06	1.00	–
1 thread	360.02	5.66	1.08	108.0
2 threads	189.00	5.74	2.06	102.9
4 threads	113.51	4.59	3.43	85.7
8 threads	76.32	1.47	5.09	63.7
16 threads	56.52	2.26	6.88	43.0
22 threads	53.68	2.19	7.24	32.9

The IQR values indicate relatively low variability for the largest matrix size. For example, the configuration with 22 threads had a median execution time of 53.68 ms and an IQR of 2.19 ms. Although some variation was observed between repetitions, the overall performance improvements remained clear.

4.2 Parallel Speedup

Figure 2 shows the speedup obtained with different thread counts.

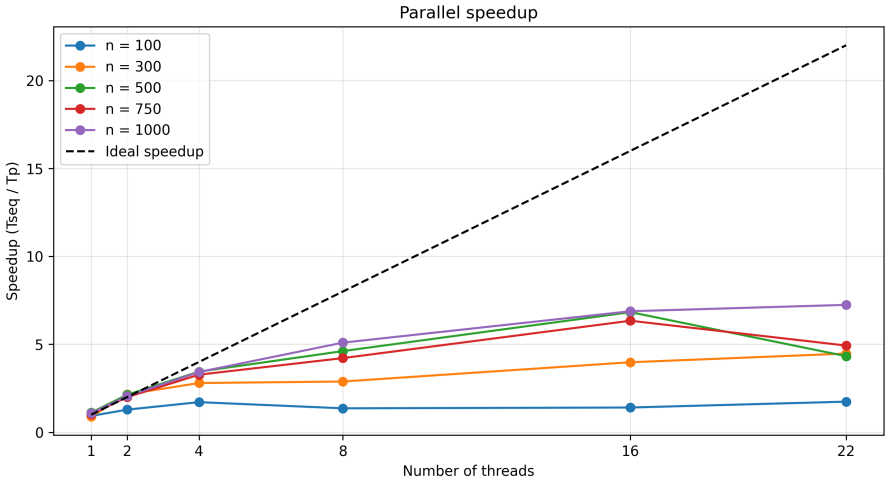


Figure 2: Parallel speedup relative to the sequential implementation.

For larger matrices, speedup generally increased as more threads were used.

The highest observed speedup was approximately 7.24 for size 1000 with 22 threads.

However, the improvement was not proportional to the number of threads. For example, increasing from 16 to 22 threads produced only a small additional reduction in execution time for the largest matrix.

The results were also not entirely consistent across sizes. For size 500, the best performance was obtained with 16 threads, reaching approximately 7.94 ms. Using 22 threads increased the time to 12.54 ms.

4.3 Parallel Efficiency

Figure 3 presents the corresponding parallel efficiency.

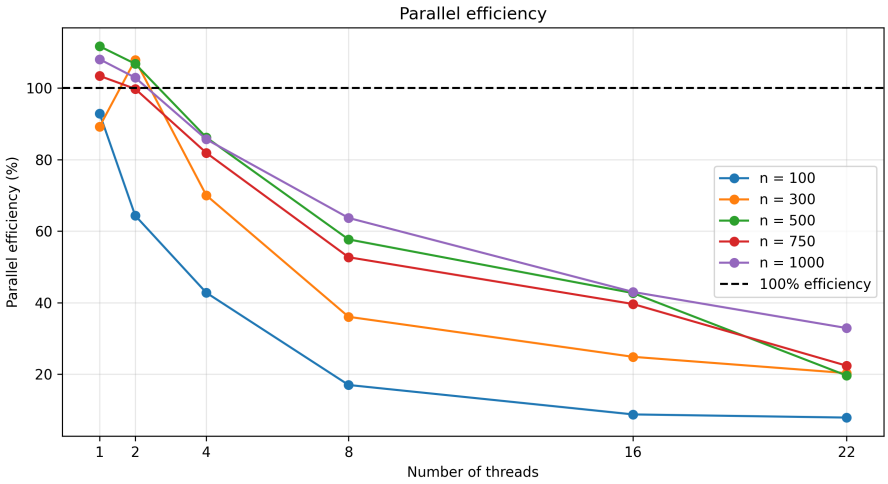


Figure 3: Parallel efficiency for different matrix sizes and thread counts.

Efficiency generally decreased as the number of threads increased.

For size 1000, efficiency was approximately 85.7% with four threads, 63.7% with eight threads and 32.9% with 22 threads.

Smaller matrices showed a more pronounced decrease, suggesting that they did not provide enough work to benefit fully from a large number of threads.

Some configurations showed efficiency slightly above 100%. These values should be interpreted cautiously, as they may reflect measurement variability, caching effects or differences in processor scheduling.

4.4 Memory Usage

Figure 4 shows the observed peak process RSS for the different configurations.

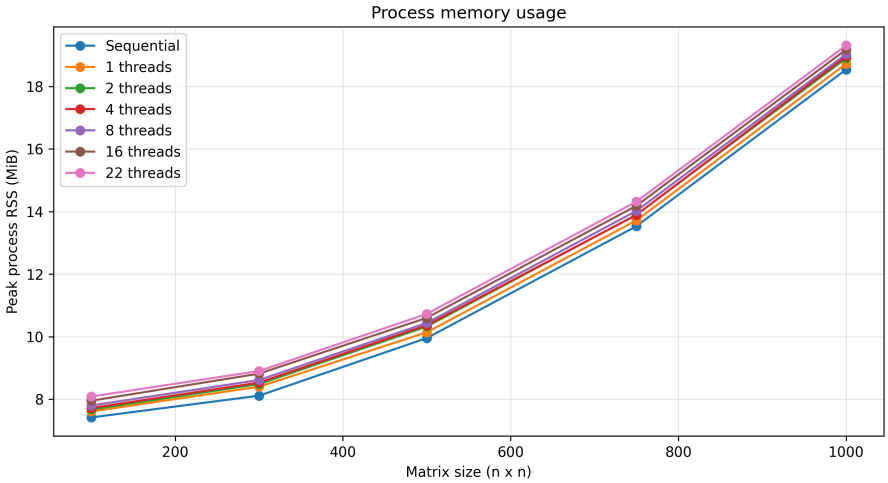


Figure 4: Peak process RSS measured for sequential and parallel multiplication.

Memory consumption increased with matrix size, following a similar trend for all implementations.

The parallel versions generally showed slightly higher RSS values, especially when more threads were used.

For size 1000, observed peak RSS was approximately 18.5–19.3 MiB across the tested configurations.

For n = 1000, the estimated array storage was 11.44 MiB, which was lower than the observed process RSS. This difference is expected because RSS also includes other memory allocations and runtime overhead. The increase in memory usage across matrix sizes was mainly associated with the storage required by the input and output matrices.

Since the threads shared the input and output matrices, parallelisation did not require additional complete copies of the data.

5 Discussion

The results indicate that OpenMP parallelisation can considerably improve dense matrix multiplication performance, particularly for larger inputs.

The main advantage comes from distributing the work between threads. Since different threads calculate separate rows of the output matrix, the implementation can perform part of the computation concurrently without conflicting writes.

However, the benefits depended on both matrix size and thread count.

For smaller matrices, increasing the number of threads did not produce consistent improvements. One possible reason is that the computational work was too limited to compensate for the overhead of parallel execution.

For larger matrices, speedup increased more clearly, although efficiency decreased as additional threads were introduced.

The comparison between 16 and 22 threads was also interesting. Although 22 threads achieved the lowest execution time for size 1000, this was not the case for sizes 500 and 750.

For size 500, execution time increased from 7.94 ms with 16 threads to 12.54 ms with 22 threads. A similar result appeared for size 750, where execution time increased from 25.14 ms to 32.33 ms.

One possible explanation is the hybrid architecture of the processor, which combines cores with different performance characteristics. The default OpenMP scheduling and the absence of explicit processor affinity may have influenced how the work was distributed. However, the experiment did not measure the actual placement of threads on individual cores, so this remains a possible explanation rather than a confirmed cause.

Some configurations showed speedup values slightly above the number of threads. To investigate this, the parallel implementation with one thread was also used as an alternative reference.

For size 1000, the sequential baseline required approximately 388.9 ms, while the parallel implementation with one thread took 360.0 ms. Using two threads reduced execution time to 189.0 ms, giving a speedup of 2.06 relative to the sequential baseline, but only 1.90 relative to the one-thread version.

A similar pattern appeared for size 500. This suggests that the apparent superlinear speedup was partly caused by differences between the two reference implementations rather than an improvement beyond ideal parallel scaling.

The one-thread results did not reveal a consistent OpenMP overhead. For smaller matrices, the parallel version was slightly slower than the sequential baseline, while for larger matrices it was slightly faster. Measurement variability and processor scheduling may have influenced these differences.

The memory measurements showed that parallel execution introduced only modest additional process memory consumption in the tested configurations.

This is consistent with the shared-memory approach, although RSS includes more than the matrix arrays and does not isolate the memory used by OpenMP.

Finally, the experiments were performed on a single machine with one compiler configuration and five repetitions per case. The measurements mainly capture short-term variability within these runs.

The conclusions should therefore be limited to the tested hardware, implementations and datasets.

6 Conclusion

This assignment investigated how thread count affects the performance of dense matrix multiplication using OpenMP.

The parallel implementation reduced execution time considerably for larger matrices, achieving a maximum observed speedup of approximately 7.24 with 22 threads for size 1000.

However, adding more threads did not always improve performance. For some configurations, particularly smaller matrices, the additional parallelism provided little benefit or even increased execution time.

Parallel efficiency generally decreased with thread count, while memory consumption remained relatively similar across the implementations.

Overall, the experiment showed that shared-memory parallelisation can improve matrix multiplication performance, but the advantage depends on the workload and available hardware.

Further experiments could investigate processor affinity, different scheduling strategies and larger matrices to better understand the limits of parallel scaling.

Use of Artificial Intelligence

ChatGPT (OpenAI) was used to assist with code development, debugging, experimental analysis and report writing. Claude (Anthropic) was also used to review the report and suggest improvements.

All experiments were executed and validated locally. The measurements were obtained from the actual runs, and the author was responsible for reviewing the results and interpreting the findings.

References

[1] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*. Available: `https://www.openmp.org/spec-html/5.2/openmp.html`

[2] Intel, *Intel Core Ultra 7 Processor 155H Specifications*. Available: https://www.intel.com/content/www/us/en/products/sku/236847/intel-core-ultra-7-processor-155h-24m-cache-up-to-4-80-ghz/specifications.html

[3] psutil Developers, *psutil Documentation*. Available: https://psutil.readthedocs.io/en/latest/